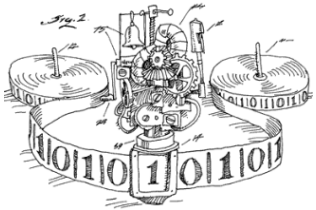
Teoría de la Computación y Lenguajes Formales



Teoría de la Computación y Lenguajes Formales: Sesión 4. Eventos Discretos, Ontologías y Composición

Édgar Chacón Ramírez
Departamento de Computación
Semestre 2026 - B

Alcance de la sesión 4

UNIVERSIDAD DE LOS ANDES VENEZUELA

La Pregunta Ontológica

Sistemas a Eventos Discretos

Composición y Cooperación

Verificación del Flujo Global

De la Ontología a la Implementación: YAML

Conclusión

La Pregunta Ontológica: ¿Qué es cada cosa?

De la Red de Petri a la Ontología: dos preguntas distintas

UNIVERSIDAD DE LOS ANDES VENEZUELA

Hasta aquí: ¿es válida esta secuencia?

Gramáticas, BNF y Redes de Petri responden todas la **misma familia de pregunta**: dado un comportamiento, ¿pertenece al lenguaje de comportamientos válidos del proceso? La Red de Petri simplemente la responde mejor, porque ya modela Producto (fichas), Proceso (transiciones) y Recursos (lugares compartidos) a la vez.

Ahora: ¿qué es cada cosa?

Cambiamos de pregunta. Ya no preguntamos si una secuencia de eventos es válida – preguntamos:

- ¿Qué **es** un "Molino"? ¿Qué atributos tiene?
- ¿Qué holones lo **contienen**? ¿Con qué otros recursos es **intercambiable**?
- ¿Cómo se **relacionan** entre sí los conceptos del dominio?

Esta es una pregunta de **ontología / modelado de datos**, no de lenguaje formal.

Por qué BNF no alcanza aquí – y no es una falla

BNF nunca fue diseñado para esto: describe conjuntos de **cadenas**, no jerarquías de **conceptos** con atributos y relaciones. No hay una "gramática Tipo -1" escondida que resuelva esto – es, sencillamente, un problema de otra naturaleza.

Representación Unificada de la Ontología

UNIVERSIDAD DE LOS ANDES VENEZUELA

El concepto de Homogeneidad

Si queremos un motor genérico capaz de simular y controlar la planta/negocio, la descripción del **Proceso**, del **Recurso** (con sus atributos y sub-procesos locales) y del **Producto** (con sus especificaciones) deben escribirse bajo la misma ontología común.

Sintaxis y Metalenguaje

- **YAML** es simplemente la sintaxis legible que usamos para escribir esa ontología homogénea de manera estructurada.
- Al igual que **PNML** (Petri Net Markup Language) describe de forma estándar una Red de Petri, el archivo **YAML** describe simultáneamente los datos y el comportamiento del sistema.

Sistemas a Eventos Discretos (DES)

¿Qué es un Sistema a Eventos Discretos?

UNIVERSIDAD DE LOS ANDES VENEZUELA

Definición de DES

Un **Sistema a Eventos Discretos (DES)** es un sistema dinámico con un espacio de estados discreto (no continuo) cuya evolución es provocada por la ocurrencia de eventos puntuales asíncronos en instantes específicos.

Comparación de Dinámicas

Sistemas Continuos (Física):

- El estado cambia de forma suave y continua sobre el tiempo.

$$\dot{x}(t) = f(x(t), u(t))$$

- Gobernado por ecuaciones diferenciales.

Sistemas Discretos (DES / Workflows):

- El estado permanece constante hasta que ocurre un evento discreto.

$$x_{k+1} = \delta(x_k, e) \quad (e \in \Sigma)$$

- Gobernado por transiciones lógicas.

Descripción Formal de un DES ...

UNIVERSIDAD DE LOS ANDES VENEZUELA

Componentes principales

En relación con un Flujo de Trabajo tenemos:

Componente	Descripción
Estado	Conjunto de variables que describen el sistema en un momento dado. El espacio de estados es discreto
Evento	Suceso que ocurre en un instante de tiempo y provoca un cambio en las variables de estado del sistema
Entidad	Actor u objeto del sistema que genera y/o recibe eventos. El estado del sistema es el agregado de los estados de sus entidades.
Flujo de Trabajo	Secuencia de eventos que pertenecen a una entidad y que cierran un ciclo funcional, desarrollándose en un intervalo de tiempo no puntual.

Descripción de la dinámica

$$DES = (X, \Sigma, f_t, x_0, X_m)$$

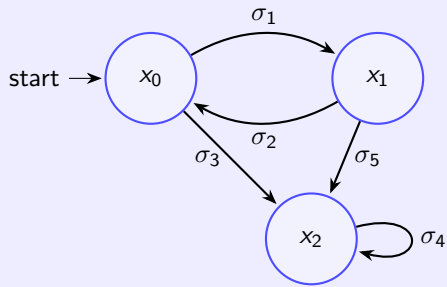
donde:

- X es el conjunto de estados discretos
- Σ Conjunto de eventos (alfabeto de eventos).
- $f_t : X \times \Sigma \mapsto X$, función de transición de estado (puede ser parcial)
- x_0 es el estado inicial.
- $X_m \subseteq X$ es el conjunto de **estados marcados** (éxito / terminación del flujo).

Representación Gráfica del DES

Grafo de Transición de Estados

El comportamiento dinámico se visualiza como un autómata, donde los nodos son los estados discretos (X) y los arcos dirigidos son las transiciones (f_t) activadas por eventos (Σ).



Aquí, $\sigma_i \in \Sigma$ representan los **eventos** puntuales que disparan los cambios de **estado**.

La Planta (G) y el Lenguaje Marcado

UNIVERSIDAD DE LOS ANDES VENEZUELA

El Lenguaje Generado $L(G)$

Representa todas las secuencias físicamente posibles de eventos que la planta (los recursos) puede realizar, incluyendo trayectorias no deseadas o fallas.

El Lenguaje Marcado $L_m(G) \subseteq L(G)$

Representa el subconjunto de secuencias de eventos que llevan a estados marcados (X_m). En workflows, un estado marcado representa el éxito de un proceso o un **Producto Terminado** (valor añadido).

La meta de la automatización

Asegurar que la evolución del sistema permanezca dentro del comportamiento deseado y alcance de forma segura el lenguaje marcado $L_m(G)$.

Eventos Controlables y No Controlables

UNIVERSIDAD DE LOS ANDES VENEZUELA

Partición del Alfabeto

Para poder **controlar** el DES (no solo describirlo), el conjunto de eventos se particiona en dos subconjuntos disjuntos:

$$\Sigma = \Sigma_c \dot{\cup} \Sigma_{uc}$$

Controlables (Σ_c)

Eventos que un agente de control puede **prevenir** o **habilitar** a voluntad. Ej: iniciar una operación, abrir una válvula, despachar una orden.

No controlables (Σ_{uc})

Eventos que ocurren de forma espontánea en la planta y **no pueden bloquearse**. Ej: fin de una operación, falla de un sensor, llegada de una pieza.

¿Por qué importa?

Un supervisor solo puede actuar deshabilitando eventos de Σ_c; nunca puede impedir que ocurra un evento de Σ_{uc}. Esta restricción es la base de toda la teoría de control supervisorio.

El Supervisor: Generador vs. Reconocedor

UNIVERSIDAD DE LOS ANDES VENEZUELA

Dos roles distintos

- La **planta** G es un **generador**: produce, por su propia dinámica física, todas las secuencias de $L(G)$ – incluyendo las indeseadas.
- El **supervisor** S es **reconocedor** ya que observa la secuencia de eventos que va generando G y, *evento a evento, decide cuáles eventos de* Σ_c *habilita para el siguiente paso* (reconoce y controla).

La Composición S/G

El comportamiento efectivo de la planta bajo supervisión es la composición síncrona S/G (el mismo operador $\|$ que formalizaremos en la sección siguiente, aplicado aquí entre supervisor y planta). Sus lenguajes cumplen $L(S/G) \subseteq L(G)$ y $L_m(S/G) \subseteq L_m(G)$.

Nota

Esta es exactamente la pareja S/G que usaremos más adelante en la condición de no-bloqueo: $\overline{L_m(S/G)} = L(S/G)$.

Composición Síncrona y Handshake

La Visión Multi-Agente / Holónica

En lugar de tener un motor centralizado que orquesta la planta, **cada recurso cuenta con su propio motor local de estados**.

La dinámica global de la planta emerge de la interacción cooperativa y el paso del producto de un recurso a otro.

La transferencia del producto

La evolución ocurre cuando dos motores locales cooperantes ejecutan un **handshake síncrono** en un canal compartido.

De Supervisor Único a Red de Autómatas Comunicantes

UNIVERSIDAD DE LOS ANDES VENEZUELA

El salto: de un supervisor a muchos

S/G funciona con **un** supervisor centralizado observando **una** planta. Para una red distribuida de recursos (nuestra visión holónica), no hay un único observador: cada recurso debe vigilarse – y coordinarse – a sí mismo.

Cada motor local es Generador *y* Reconocedor a la vez

Cada autómata local R_i deja de ser solo un generador de sus propios eventos ($!e$) y pasa a ser también reconocedor de los eventos que le llegan de sus vecinos ($?e$). Esta doble capacidad – generar y reconocer simultáneamente – es precisamente lo que hace posible el **handshake** que sigue.

Autómatas Comunicantes

Una red de autómatas que son, cada uno, generador-reconocedor de su propio alfabeto local se conoce como **autómatas comunicantes** (*communicating automata*) – el modelo subyacente detrás de la composición síncrona $\|$ y el operador CCS/CSP que veremos a continuación.

El Mecanismo de Handshake

UNIVERSIDAD DE LOS ANDES VENEZUELA

Eventos de Entrada y Salida (CCS / CSP)

La comunicación entre motores se modela etiquetando los eventos:

- $!req$ / $!ack$: Evento activo de envío (salida).
- $?req$ / $?ack$: Evento pasivo de recepción (entrada).

La Acción Silenciosa τ

Cuando coinciden la transición activa $!e$ de un recurso R_1 y la receptora $?e$ del recurso R_2, el sistema conjunto realiza la transición de forma síncrona, ocultando la señal al entorno (**evento interno** τ):

$$\frac{R_1 \xrightarrow{!e} R_1', \quad R_2 \xrightarrow{?e} R_2'}{R_1 \parallel R_2 \xrightarrow{\tau} R_1' \parallel R_2'}$$

Símbolos comunes a los dos lenguajes

Esto se cumple para los símbolos que son comunes a los lenguajes de las dos máquinas

El Operador de Composición Síncrona ($\|$)

UNIVERSIDAD DE LOS ANDES VENEZUELA

Construcción del Espacio de Estados Global

El comportamiento combinado de la red de recursos se formaliza mediante la composición síncrona $(G_1 \parallel G_2 \parallel \ldots \parallel G_n)$.

La Explosión de Estados

El número de estados globales crece de forma exponencial con el número de recursos $(|Q_{global}| \approx \prod |Q_i|)$.

Esto hace inviable analizar el sistema a mano y justifica la necesidad de utilizar herramientas de software (simuladores e intérpretes) para verificar su comportamiento de forma automática.

Verificación:
¿Aseguramos que el producto termine?

Propiedad de No-Bloqueo (Deadlock-Free)

UNIVERSIDAD DE LOS ANDES VENEZUELA

¿Qué es el Deadlock (Bloqueo Mutuo)?

Un estado global donde todos los motores locales están esperando una acción de los demás, quedando el sistema completamente paralizado.

Caso típico en manufactura

La máquina 1 espera a que la máquina 2 reciba la pieza, pero la máquina 2 está esperando a que la máquina 1 se libere. Ninguno puede avanzar.

Definición Formal de No-Bloqueo

El sistema es libre de bloqueo si desde cualquier estado alcanzable siempre es posible llegar a un estado marcado de éxito:

$$\overline{L_m(S/G)} = L(S/G)$$

Propiedad de Livelock (Lazos Infinitos)

UNIVERSIDAD DE LOS ANDES VENEZUELA

¿Qué es el Livelock?

El sistema global sigue realizando transiciones de eventos de forma continua (no está congelado), pero queda atrapado en un bucle local sin la posibilidad de progresar hacia el estado marcado final de terminación del producto.

Caso en procesos de negocio

Un flujo de aprobación de facturas donde el documento rebota infinitamente entre "Verificar" y "Rechazar" por discrepancias de céntimos, sin llegar jamás a la aprobación o al rechazo definitivo.

El Intérprete como Validador de la Terminación

UNIVERSIDAD DE LOS ANDES VENEZUELA

¿Cómo se verifica en la práctica?

Los estudiantes programarán un cargador del YAML que simule la evolución del token de producto a través de la red de recursos.

El motor evaluará mediante algoritmos de búsqueda en grafos (ej: BFS/DFS) si todas las trayectorias posibles del token de producto convergen hacia el **Estado Marcado de Éxito**, garantizando la terminación segura del proceso antes de iniciar la ejecución real en la planta.

YAML: El Metalenguaje del DES

Mapeo Directo de la Teoría a la Implementación

La ontología en YAML no es arbitraria: cada campo está diseñado para reflejar exactamente los componentes del DES:

Concepto DES	Símbolo	Correspondencia en YAML
Estados	X	Lista `states` (ej: `idle`, `processing`, `done`)
Eventos	Σ	Lista `events` (ej: `start`, `finish`, `fail`)
Función de Transición	$f_t : X \times \Sigma \to X$	Array `transitions`: {`from`, `event`, `to`}
Estado Inicial	x_0	Campo `initial_state`
Estados Marcados	$X_m \subseteq X$	Booleano `marked: true` en el estado correspondiente
Controlabilidad	Σ_c / Σ_{uc}	Atributo `controllable: true/false` en cada evento

La Clave de la Homogeneidad

Tanto el **Recurso** (máquina), como el **Producto** (pieza) y el **Proceso** (flujo general) comparten **exactamente la misma estructura** de estados y eventos. Esto permite que el motor de simulación trate a todos por igual.

Ejemplo en YAML: El Recurso “Molino”

Código del autómata local del Molino

```
resource
  id "Molino_001"
  type "Maquina_CNC"
  # Estados (X)
  states
    - id "idle"
      marked false
    - id "clamping"
      marked false
    - id "milling"
      marked false
    - id "finished"
      marked true # <-- Estado marcado (X_m)
    - id "error"
      marked false
  # Estado inicial (x0)
  initial_state "idle"
  # Eventos (Sigma) con controlabilidad
  events
      - id "start_milling"
        controllable true # El supervisor puede bloquearlo
      - id "part_loaded"
        controllable false # Evento no controlable (llega pieza)
      - id "cycle_done"
        controllable false # La maquina avisa sola
      - id "fault_detected"
        controllable false # Evento no controlable (falla)
      - id "reset"
        controllable true

  # Transiciones (f_t)
  transitions
      - from "idle"
        event "part_loaded"
        to "clamping"
      - from "clamping"
        event "start_milling"
        to "milling"
      - from "milling"
        event "cycle_done"
        to "finished"
      - from "milling"
        event "fault_detected"
        to "error"
      - from "error"
        event "reset"
    to "idle"
```

Handshake en YAML: Canales de Comunicación

Definiendo la Interfaz (! y ?)

Para que dos recursos cooperen (paso de producto), se definen **canales**. Emisor y receptor deben compartir el mismo `id` de canal y el mismo nombre de evento – exactamente como exige la regla de sincronización $R_1 \xrightarrow{!e} R_1', R_2 \xrightarrow{?e} R_2'$:

```yaml
# Recurso A (Molino) — Emite la pieza al terminar
  channels
    - id "conveyor_AB"
      direction "output" # Equivale a !e en CCS
      event "part_transfer"

# Recurso B (Torno) — Recibe la pieza
  channels
    - id "conveyor_AB"
      direction "input" # Equivale a ?e en CCS
      event "part_transfer"
```

Regla de Sincronización en el Motor

Cuando `Molino` ejecuta `!part_transfer` en `conveyor_AB` y `Torno` está listo para `?part_transfer` en ese mismo canal, el motor dispara la transición τ (oculta). Si alguno no está listo, el producto no avanza (espera o bloquea).

El Secreto del Acoplamiento

Especificación del Producto en YAML

La Receta: El Lenguaje que debe alcanzar $L_m(G)$

El producto no es una pieza pasiva; tiene una **ruta obligatoria** (workflow) que define el lenguaje marcado deseado:

```
product
  id "Pieza_XYZ_2026"
    attributes
      material "Aluminio_7075"
          tolerance "0.01mm"
          batch "Lote_42"

# Aqui definimos el "Lenguaje Marcado" (X_m deseado)
    required_route
        - resource "Molino_001"
          operation "Desbaste"
              expected_output_state "finished"
        - resource "Torno_002"
              operation "Acabado"
              expected_output_state "finished"
        - resource "Calibre_003"
              operation "Inspeccion"
              expected_output_state "approved"

      # Atributos de verificacion (para el validador)
      max_cycle_time 120 # segundos
```

¿Cómo se verifica?

El intérprete carga la ontología, compone los autómatas de los recursos mediante $\|$, y ejecuta un **model**

Conclusión de la Sesión 4

UNIVERSIDAD DE LOS ANDES VENEZUELA

Resumen conceptual

- Pasamos de verificar la validez de cadenas sintácticas a estructurar una ontología de conceptos (YAML).
- Modelamos la planta bajo el formalismo de **Sistemas de Eventos Discretos (DES)**.
- Analizamos cómo la interacción cooperativa local de recursos mediante **handshakes** define el comportamiento global.
- Establecemos que la ausencia de bloqueos (*deadlocks*) y la garantía de terminación son propiedades demostrables mediante el análisis de la composición síncrona.

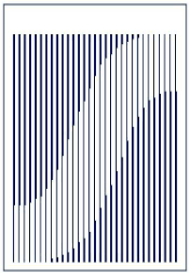
Frase de Cierre

El modelado formal de la planta y sus recursos nos permite simular el futuro del proceso, previniendo los bloqueos en el código de diseño antes de que ocurran en la fábrica.

¡Muchas gracias!



UNIVERSIDAD
DE LOS ANDES
VENEZUELA



Édgar Chacón Ramírez
email: echacon@ula.ve